Lumora - Google AI Studio Master Build Prompt (Android)

Instruction document for the Google AI Studio coding agent. Read the whole document, then follow it exactly, stage by stage (section V). Compile and test at the end of every stage.

ROLE AND OPERATING RULES

You are a senior Android engineer building a complete, production-quality native Android app called **Lumora** (working title; keep the name in one constant `AppConfig.APP_NAME`). You are in Google AI Studio Build mode, Android target. Build **real, working functionality**. No static mock-ups, no fake data, no random scores, no dead buttons.

Rules of engagement

1. **Inspect first.** Before changing anything, read the existing project (build files, manifest, existing packages) and keep one consistent architecture. Do not create a second module or a second activity.
2. **Work in the 9 stages in section V.** Finish a stage, **compile**, fix all errors and warnings that matter, run the unit tests you wrote for that stage, then print a short `STAGE REPORT` (what works, what is stubbed, what needs manual setup) and go to the next stage. Never dump the whole app in one unverified pass. Never break an earlier stage.
3. **First deliverable priority:** a fully working core (Stages 1–4): auth, onboarding, consent, capture, face measurements, results, history, compare, settings, delete. Advanced features come after, behind the same architecture.
4. **Stack (fixed):** Kotlin, Jetpack Compose, Material 3, single Activity, single Gradle module, MVVM + repository pattern, Kotlin Coroutines + `StateFlow`, Navigation Compose, Room, DataStore, WorkManager, CameraX, ML Kit, Firebase (Auth, Firestore, Storage, AI Logic, App Check), Google Play Billing KTX, Credential Manager (`androidx.credentials` + `googleid`). Use Hilt **only if** it compiles cleanly in this environment; otherwise use a simple manual `AppContainer` (service-locator object created in `Application`). Pick current stable versions through the project's version catalog; **do not guess versions: use what resolves**.
5. **Hard constraints:** no NDK/C++; no Java; no XML layouts (the manifest and resource XML required by Android are fine); no separate Node/server code; no private API keys, tokens, passwords or test accounts anywhere in code, resources or assets; all Gemini calls go through **Firebase AI Logic** with **App Check**.
6. **If a dependency is unavailable or incompatible:** say so in the stage report, use a verified alternative from the fallback list in section K, and keep the interface so the implementation can be swapped.
7. **Honesty rules:** where a capability truly needs a server or the Play Store (purchase verification, server-side quotas, real purchases), implement the client part, label the rest `PENDING BACKEND` in code comments and in the stage report, and show a visible "unverified" state in the UI. Never claim a test passed unless you ran it.
8. **Product ethics (non-negotiable):** the app describes photos, not people. No attractiveness "tier", no ranking of looks, no "dimorphism"/masculinity/femininity score, no comparison with other people, no celebrity look-alike, **no face recognition or identity matching of any kind**, no face-warping simulation. No shame-based wording anywhere ("ugly", "bad", "worth working on", "flaw"). Do not suggest a score reflects a person's worth. Do not diagnose any medical condition from photos. Cosmetic suggestions are separate from medical advice.
9. Write clean, documented, testable code: small composables, state hoisting, immutable UI state classes, `sealed interface` for UI events/results, no business logic inside composables, no `GlobalScope`, no blocking on main thread, all user-visible strings in `strings.xml` (so the app can be localised).

A. PRODUCT

Lumora is a privacy-first face and skin *progress journal*. The user takes a guided front photo (optionally a side photo) under consistent conditions. On the phone, the app measures face proportions from facial landmarks and reads skin appearance from the photo. It shows measurements as *positions within typical ranges with visible uncertainty*, explains what can and cannot change, tracks progress over time, offers an evidence-tagged routine and an AI coach, and keeps photos private by default.

Positioning line: "See how your face and skin photograph over time — privately, and without being told what you're worth."

Free vs Pro (all limits live in one `EntitlementPolicy` data class so the owner can tune them):

- Free: unlimited on-device scans, measurements, results, local history, compare, plan/habits, reminders, export, delete. AI skin read: 1 per 7 days. Coach: 5 messages/day.
- Pro: AI skin reads up to 10/day, coach up to 100 messages/day, optional cloud photo backup, weekly AI report, PDF summary export, advanced compare (all metrics + trends).
- Never lock a result the user already captured.

B. DESIGN SYSTEM (implement as `ui/theme/`)

Colour tokens (define a `LumoraColors` object and map to Material 3 `ColorScheme`; light / dark):

- background `#F7F5F2` / `#0F1214`; surface `#FFFFFF` / `#171B1E`; surfaceVariant `#EDEAE4` / `#222729`
- onSurface `#16181D` / `#ECEFF1`; onSurfaceMuted `#5B626B` / `#9AA3AB`
- primary `#2F5D62` / `#7FC4C0`; onPrimary `#FFFFFF` / `#08201F`; accent `#D99A2B` / `#E8B04C`
- success `#3A7D44` / `#7BC48A`; caution `#9A6A12` / `#E0B15A`; error `#B3261E` / `#F2B8B5`; outline `#D6D2C9` / `#343B3F`
- Follow system dark mode; add an Appearance setting (System / Light / Dark) stored in DataStore.
- Never convey status by colour alone; always add an icon and text.

Typography: Plus Jakarta Sans (UI) and Fraunces (display/numerals) via `androidx.compose.ui:ui-text-google-fonts` downloadable fonts; if the provider is unavailable, fall back to `FontFamily.SansSerif` / `Serif` without crashing. Scale: Display 32/40 · Headline 24/32 · Title 18/24 · Body 16/24 · Label 13/18 · Caption 12/16; weights 400/500/600; tabular numerals in charts.

Shape/spacing/elevation: corner radii 12 (inputs, chips), 20 (cards), 28 (sheets, centre action). 4-dp grid (4, 8, 12, 16, 24, 32, 48). Gutters 16 dp compact, 24 medium, 32 expanded. Tonal elevation; shadows 0–2 dp only.

Motion: 180–280 ms, standard easing; spring for result reveal; chips cross-fade. Read `Settings.Global.ANIMATOR_DURATION_SCALE`; when it is 0, skip animations.

Layout/responsive: use `WindowSizeClass`. Compact: bottom navigation. Medium/expanded: `NavigationRail`, and two-pane Results (list + detail) at ≥ 840 dp. Support portrait phones, landscape, tablets and foldables. All screens scroll if content overflows; no clipped text at 200% font scale.

Reusable components (in `ui/components/`): `LumoraTopBar`, `PrimaryButton`, `SecondaryButton`, `QuietTextButton`, `GateChip`, `IndexRing`, `RangeBar`, `MetricRow`, `InsightCard`, `PrivacyBadge`, `QualityChip`, `PhotoFrame`, `SectionHeader`, `ListSwitchRow`, `EmptyState`, `ErrorState`, `LoadingSkeleton` (shimmer, respects reduced motion), `ConfirmDialog`, `AppBottomSheet`, `StreamingMessageBubble`, `HabitRow`, `TrendChart` (Canvas-drawn line chart with accessible text summary), `BeforeAfterSlider`, `OfflineBanner`, `UnverifiedBadge`.

Accessibility: 48-dp minimum touch targets; `contentDescription` or `clearAndSetSemantics` on every icon/image; `heading()` semantics; focus order matches reading order; TalkBack announces capture-gate changes through `LiveRegionMode.Polite`; contrast ≥ 4.5:1 text, ≥ 3:1 UI; supports font scale to 200%; haptic feedback optional via setting; no flashing content.

C. NAVIGATION AND ROUTING

Single `MainActivity` with `NavHost` (type-safe routes via sealed class `Route`). `AppNavGraph` observes `SessionState` (`Loading | SignedOut | NeedsOnboarding | NeedsConsent | Ready`) and routes accordingly. Routes: `splash`, `welcome`, `authChoice`, `signUp`, `signIn`, `forgotPassword`, `verifyEmail`, `onboarding/{step}`, `consent`, `notificationRationale`, `main` (bottom bar: `today`, `progress`, `coach`, `you` plus centre action `scanHub`), `scanHub`, `cameraRationale`, `capture/{kind}` (`front|side`), `review`, `processing`, `results/{scanId}`, `metric/{scanId}/{metricId}`, `overlay/{scanId}`, `skin/{scanId}`, `scanDetails/{scanId}`, `compare`, `chat/{threadId}`, `chatHistory`, `plan`, `routine`, `weeklyCheckIn`, `settings`, `privacyCentre`, `export`, `deleteAccount`, `paywall/{reason}`, `subscription`, `legal/{doc}`, `notificationSettings`, `appearance`, `about`. **Back behaviour:** system Back pops one level; from a bottom-bar tab it returns to `today`; from `today` it exits. Capture/processing screens confirm before discarding. Process death must restore the current route and any in-progress scan id (use `SavedStateHandle` and persisted scan draft in Room).

D. SCREENS (build every one; every button and input must work)

Common to all screens: loading skeleton, empty state, error state with a retry action, offline banner when relevant, system-bar insets handled, previews for light/dark.

N01 Splash — brand mark; resolves session, consent version and onboarding flags; goes to the right route within 1.5 s or shows retry on failure.

N02 Welcome — 3-page pager: (1) honest promise, (2) "Photos stay on your phone unless you choose otherwise", (3) "Describes photos, not people". Buttons: Get started → N03; Sign in → N05. Page dots with semantics.

N03 Auth choice — "Continue with Google" (Credential Manager `GetGoogleIdOption`, nonce, then `GoogleAuthProvider.getCredential` → Firebase), "Continue with email" → N04, links to Privacy and Terms.

N04 Sign up — email, password (show/hide, strength meter, min 8 chars), confirm age 13+ checkbox, accept terms checkbox. Inline validation, disabled button until valid, Firebase `createUserWithEmailAndPassword`, send verification email, then N07. Errors: email in use, weak password, no network, generic.

N05 Sign in — email, password, "Forgot password?" → N06, Sign in; Google button too. Errors mapped to friendly text; lock the button while loading.

N06 Forgot password — email → `sendPasswordResetEmail`; always show the same neutral confirmation (no account enumeration).

N07 Verify email — explains, "Resend email" (30 s cooldown), "I've verified" (reloads user), "Continue for now" (non-blocking, with a persistent gentle banner on Today until verified).

N08 Profile (4 steps, progress bar, Back works, values saved each step) — (1) display name; (2) birth year (optional, picker; if age < 18 enable **Teen mode**: hide numeric indices by default, hide active-ingredient suggestions, show a "talk to a parent/dermatologist" note; block under 13 with a kind explanation and sign-out); (3) skin type (dry, oily, combination, normal, sensitive, not sure) and concerns (multi-select: acne, marks, redness, dryness, oiliness, texture, dark circles, fine lines); (4) optional height (cm/ft-in) and "Reference ranges" preference (Prefer not to say [default] / Female-typical / Male-typical / Neutral). Explain: this only changes which published typical ranges are shown for a few measurements and is never scored on gender.

N09 Consent centre — separate switches with plain explanations and "Learn more": (a) *On-device face analysis* (required to scan), (b) *Cloud AI skin & coach analysis* (sends a cropped face photo / text to Google Gemini via Firebase AI Logic; off = skin read is on-device only and the coach is disabled), (c) *Cloud photo backup* (Firebase Storage, off by default), (d) *Anonymous analytics* (off by default; if you implement none, hide this switch and say "No analytics is collected"). Store `consentVersion`, timestamp and per-scope booleans in Firestore and DataStore. Consent can be changed any time in N33. Never pre-tick (b)–(d).

N10 Notification rationale — explains reminders, asks `POST_NOTIFICATIONS` on API 33+, time picker for weekly scan reminder; "Not now" works.

N11 Today — greeting; offline/verify banners; latest-result card (opens N17); next-scan card with days since last scan; today's habits (`HabitRow` toggles, persisted); streak chip with one weekly rest day; quick actions (Scan, Ask coach, Plan). Empty state: "Take your first scan".

N12 Scan hub — what to expect (2 min), checklist (soft light facing a window, neutral expression, hair off forehead, no glasses/hat, phone at eye level at arm's length), buttons: *Front scan*, *Side scan (beta)*, *Choose from gallery*. A `PrivacyBadge` states what stays on the phone.

N13 Camera rationale — explains why the camera is needed; Allow → system prompt; if permanently denied → "Open settings" and "Use gallery instead".

N14 Live capture — CameraX front camera `Preview` + `ImageAnalysis` (`STRATEGY_KEEP_ONLY_LATEST`, ~640×480) + `ImageCapture`. Overlays: oval guide, five `GateChips` (Head angle, Light, Distance, Expression, Steady) turning to "ready" with icon + text and a polite TalkBack announcement and light haptic. Auto-capture when all gates have held for 8 consecutive analysis frames; manual shutter always available (shows a warning that results will be less reliable if gates fail). "Mirror preview" toggle (affects preview only; saved photo is never mirrored). Torch hint on low light is text only. Front scan targets yaw/pitch ≈ 0; side scan targets yaw 70–90° and shows a profile guide.

N15 Photo review — shows the final photo, accept / retake; for gallery photos runs the same checks and explains *exactly* which check failed (no face, more than one face, too dark, blurry, face too small, head turned, eyes closed, expression, cropped) with one retake tip. Notes that location and other hidden

metadata are removed.

N16 Processing — staged progress ("Finding your face on this phone → Measuring proportions → Reading skin appearance"), cancel button, timeouts (30 s per cloud call), failure states with Retry and "Keep photo on this phone".

N17 Results — top `QualityChip` (photo quality 0–100 with reasons); `PrivacyBadge` panel "What stayed on your phone / what was sent" (listing actual actions taken in this scan); group **indices** (0–100, labelled "photo-based estimate"): *Proportions*, *Symmetry*, *Skin clarity* — **no single overall score**; measurement list (`MetricRow` with `RangeBar`, neutral wording: "Within typical range / Near typical range / Outside typical range — this is normal variation", changeability tag); insight cards: "What changed since your last scan" (only when change exceeds the noise floor, else "within normal photo-to-photo variation"). If Teen mode or "Hide numbers" is on, show words and bars only. Actions: Save note, Compare, Share (user-initiated, image excluded by default), Delete scan.

N18 Measurement detail — definition in plain words; landmark overlay for this metric; `RangeBar` with provisional-range label and source note; uncertainty (mean ± SD across the burst); changeability tag (**Structure / Soft tissue / Presentation / Skin**); "What affects this reading" (head angle, lens distance, expression, hair, light) and "What can help" (grooming, posture, light, habits) vs "mostly structure"; history sparkline; disclaimer line.

N19 Overlay viewer — pinch-zoom, toggle landmark groups (eyes, nose, mouth, jaw, symmetry axis); overlay drawn from stored normalised landmarks that are **recomputed from the photo when needed and never uploaded**.

N20 Skin results — descriptors with confidence (e.g., "Redness: mild, medium confidence"), "Skin clarity index", routine suggestions linking to N29, evidence tags (Strong / Moderate / Limited) on each tip, "This is a cosmetic photo reading, not a medical assessment. See a doctor or dermatologist for any concern." Painful, deep, bleeding, changing moles → explicit "see a clinician" card.

N21 Side profile (beta) — same capture gates but pose-adjusted; results unscored with "Approximate" banner and accuracy tag; shows convexity, nasolabial angle, E-line relationships only when yaw 70–90° and landmark confidence is adequate; otherwise explains the retake.

N22 Progress — period selector (4 wk / 12 wk / all); `TrendChart` per index with text alternative; calendar of scan days; scan list (photo thumbnail only if retained); empty state; each point opens N23.

N23 Scan details — date, engine version, quality flags, note field, delete scan, view results/overlay.

N24 Compare — pick any two scans (same kind), `BeforeAfterSlider` (needs retained photos; otherwise shows metric-only compare), per-metric deltas with the noise-floor rule and wording "meaningful change / within normal variation".

N25 Coach chat — streaming replies from Gemini (Firebase AI Logic), markdown rendering (bold, lists, links disabled), suggestion chips ("How do I build a simple routine?", "What affects my symmetry reading?"), message input with send, stop-generation, copy, report (N27), daily-limit meter and paywall at limit, offline state, retry on failure. Disabled with an explanation if consent (b) is off.

N26 Chat history — threads list, rename, delete one, delete all (confirm). Stored in Room only.

N27 Report reply sheet — reason chips (Unsafe, Inaccurate, Rude/shaming, Medical overreach, Other) + optional note; writes to `users/{uid}/replyReports` with the reply text; confirms; never blocks.

N28 Plan — up to 3 focus areas (Skin, Grooming, Habits, Posture/Sleep) chosen from the latest results; targets framed as intentions ("a goal, not a prediction"); add/remove/reorder actions; copy states that bone-structure ratios are not changed by habits.

N29 Routine builder — AM/PM steps assembled from **generic ingredient templates** (cleanser, moisturiser, sunscreen, plus optional actives for adults only) tailored to skin type/concerns, with evidence tags and patch-test and "ask a pharmacist/dermatologist" notes. **No brand names, no affiliate links.** Note that prescription status varies by country.

N30 Weekly check-in — 3 quick questions (sleep, routine consistency, mood about appearance 1–5). If the mood answer is low or the user writes distress about appearance, show a gentle supportive message and a "talk to someone you trust or a professional" card; never push the scan.

N31 You — name, email, plan status, shortcuts to Settings, Privacy Centre, Subscription.

N32 Settings — Appearance, Notifications, Privacy centre, Subscription, Legal, About, Sign out (confirm; clears session-bound data per section R).

N33 Privacy centre — consent switches (same as N09), photo retention (`Don't keep photos` [default] / `Keep on this device` / `Back up to my account for 30 / 90 / 365 days or until I delete`), "Delete all my photos now", "Hide numbers", "Block screenshots on photo screens" (`FLAG_SECURE`, default on), "App lock" (BiometricPrompt, device credential fallback), Export my data (N34), Delete account (N35), "What we send and where" explainer. Each switch persists and takes effect immediately.

N34 Export — builds a zip (`scans.json`, `scans.csv`, `profile.json`, `chats.json`, `habits.csv`, optional photos if retained) in app cache and offers Save (SAF `CreateDocument`) or Share; clears cache after.

N35 Delete account — explains exactly what is deleted; re-authenticate (password or Google); type `DELETE`; progress; then signs out and returns to N02. Failure leaves a resumable `deletionRequested` marker and a "Finish deletion" retry on next launch.

N36 Paywall — benefits list, price/period strings from Play `ProductDetails` only, Subscribe, Restore purchases, Not now, terms/privacy links, "Cancel anytime in Google Play". Reason-aware headline (e.g., coach limit). No fake urgency, no countdowns, no pre-selected expensive option.

N37 Manage subscription — status, renewal/expiry (from purchase data), "Manage in Google Play" deep link, restore, `UnverifiedBadge` while server verification is pending.

N38 Legal viewer — renders bundled Markdown for Privacy Policy, Terms, Methodology, Medical disclaimer (write honest, consistent texts that match the actual behaviour of this app; include version and date; the Methodology page lists every measurement, its range source, and its limits).

N39 Notification settings — channels: Weekly scan reminder, Daily habits, Weekly report; each toggle + time; quiet hours; "Open system settings" when blocked.

N40 Appearance — theme, mirror preview default, haptics on/off.

N41 Global states — `EmptyState`, `ErrorState`, `OfflineBanner`, `NotFound`; 404-safe routes.

E. DOMAIN AND FEATURE BEHAVIOUR

E1. Capture quality gates (all constants in `CaptureConfig`; these are *recommended defaults to tune on real devices*)

- Pose (ML Kit Face Detection Euler angles): |yaw| ≤ 8°, |pitch| ≤ 8°, |roll| ≤ 6° for front; side: yaw 70–90°.
- Distance: face bounding-box width 40–75 % of frame width and fully inside the frame with margin.
- Light: mean luma on face ROI 70–200 (0–255); left/right ROI luma gap < 15 %; no strong clipping (> 5 % pixels at 250+).
- Sharpness: variance of Laplacian on face ROI ≥ `sharpnessFloor` (calibrate per device class; store in config).
- Expression: `leftEyeOpenProbability`/`rightEyeOpenProbability` > 0.6, `smilingProbability` < 0.3 (enable classification mode).
- Stillness: bounding-box centre moves < 1.5 % of frame over the last 600 ms.
- Exactly one face; otherwise block with a specific message.
- When all gates hold for 8 consecutive analysis frames → collect an 8-frame landmark burst (median per landmark after One-Euro smoothing) while taking an `ImageCapture` still.
- Output `QualityReport(score 0–100, flags[], perGate details)`; score is a documented weighted function, unit-tested.

E2. Face landmark engine (`face/engine/`)

Define `interface FaceLandmarkEngine { suspend fun analyse(frame: FrameInput): LandmarkResult }`. Implementations:

1. `MlKitFaceMeshEngine` (primary): `com.google.mlkit:face-mesh-detection`. It is a beta API intended for front-facing, close selfie images; handle its limits and report `ENGINE_UNSUPPORTED` clearly.
2. `MlKitContourEngine` (fallback): `com.google.mlkit:face-detection` with `CONTOUR_MODE_ALL`.

Keep a `engineVersion` string stored with every scan. Keep engines swappable; **verify canonical index mapping with unit tests on synthetic landmark fixtures** (e.g., construct a symmetric synthetic face and assert symmetry ≈ 0). Commonly used canonical-mesh indices (verify before relying): eye outer corners 33/263, inner corners 133/362, nose tip 1 (subnasale 2), mouth corners 61/291, upper lip centre 0/13, lower lip 14/17, chin 152, forehead mesh top 10, cheek width points 234/454, jaw-side points ≈ 172/397 and

58/288, nose alae 129/358. If the primary engine is not available, say so and use the fallback with a reduced measurement set. **Never** use the face for identification or embeddings. Never persist raw landmarks beyond what N19 needs, and never upload them.

E3. Measurement engine (`face/measure/`)

Pure Kotlin, no Android dependencies, 100 % unit-tested. Unit = inter-ocular distance. Every metric returns `MetricValue(id, value, sd, unit, rangePosition, reliability, changeability)`. Catalogue (bands are **provisional**, loaded from `assets/reference_bands.json` which also holds each band's source note; show "provisional" in the UI where the note says so; do not invent citations — use these classical references only where listed):

- Facial thirds: middle-to-lower third ratio (classical equal-thirds convention; the neoclassical canons popularised by Farkas et al., 1985 — present them as conventions, not validated targets). **Upper third is not measured** (no hairline detection) — state this on the Methodology page.
- Midface width : height; eye spacing (inter-canthal distance ÷ eye width, orbital canon); nose width ÷ inter-canthal distance (nasal canon); mouth width ÷ nose width (naso-oral, ≈ 1.5 as a convention); lip ratio lower : upper.
- Symmetry: mean normalised mismatch of mirrored landmark pairs across a fitted midline (head-roll corrected); also eye-height and mouth-corner-tilt asymmetry.
- Contour (descriptive, **unscored**): jaw width ÷ cheekbone width, front jaw angle, chin height share, canthal tilt.
- Side (beta, **unscored**): facial convexity, nasolabial angle, upper/lower lip to E-line; show only when pose and confidence allow.

`rangePosition ∈ {WITHIN, NEAR, OUTSIDE, DESCRIPTIVE}`; for DESCRIPTIVE show the value and an explanation only. Indices: `Proportions` and `Symmetry` = documented transformation of in-range positions to 0–100, labelled "photo-based estimate". **Noise floor:** per-metric `minMeaningfulChange = max(config floor, 2 × SD)`; changes below are reported as within normal variation. Keep a rolling `MetricNoise` per metric from repeated scans. Changeability tags: Structure, Soft tissue, Presentation, Skin. Include a **Methodology** document (N38) generated from the same catalogue so docs and app can never disagree.

E4. Skin appearance (`skin/`)

1. On-device: convert the face crop to CIE LAB; define ROIs from landmarks (forehead, both cheeks, under-eyes; exclude brows, eyes, lips, hair) → statistics: tone evenness (local ΔL*/Δa*/Δb* variation), redness proxy (a* difference cheek vs forehead), under-eye contrast (L* difference), texture proxy (local luma variance after normalisation), shine proxy (specular fraction). Explain on screen that light, camera and make-up move these as much as skin does.
2. Cloud (only with consent b): send a **cropped, downscaled (≤ 768 px long edge, JPEG q≈85, no EXIF)** face image to Gemini via Firebase AI Logic with **structured output** (`responseSchema`) returning *categorical descriptors only*: for each of acne-like blemishes, post-blemish marks, redness, dryness/flaking, oiliness/shine, texture, under-eye darkness, fine lines → `{severity: none|mild|moderate|marked, confidence: low|medium|high}`, plus `notes[]` (plain language) and `needsClinician: Boolean` with `reason`. The model must never output a number the user sees.
3. `SkinScoreEngine` (pure Kotlin, unit-tested) deterministically maps on-device statistics + descriptors to a 0–100 **Skin clarity index** with a bounded descriptor adjustment (±10 points) and returns the components so the UI can show "why". If the cloud step is declined or fails, show on-device results only.
4. Prompt (system) for the skin call: cosmetic appearance only; never diagnose; never mention conditions by name beyond descriptive categories; if a lesion looks concerning or the image is unclear, set `needsClinician`/low confidence; ignore identity; do not estimate age, gender, ethnicity or attractiveness.

E5. AI coach (`coach/`)

- Model via `Firebase.ai(backend = GenerativeBackend.googleAI()).generativeModel(modelName = AiConfig.TEXT_MODEL, systemInstruction = …)`. Keep model names in `AiConfig`; use the current stable Flash-class model that Firebase AI Logic supports at build time; do not hard-code anything else.
- **Context sent:** the user's message, last ≤ 10 turns, profile summary (display name omitted), skin type/concerns, Teen-mode flag, the latest scan's *indices and metric positions as JSON* (no images, no landmarks), active plan, today's habits. **Never send images to the coach** unless the user explicitly attaches one for that message (then show a preview and a "sent to Google" notice).

- **System prompt requirements:** cosmetic and habit guidance only; no diagnosis, no medication/prescription dosing, no tier/attractiveness ranking, no comparison with others, no celebrity matching, no identity inference; distinguish *evidence level* (Strong / Moderate / Limited); say what is bone structure vs changeable; suggest a doctor/dermatologist for red flags; for distress about appearance, self-harm cues or eating-disorder cues respond with care, avoid numbers/targets, and encourage talking to a trusted person or professional (do not provide diet/exercise numbers in that case); Teen mode: no active ingredients, no numeric indices; refuse requests to rate "how attractive" the user is and offer a constructive alternative.
- Streaming (`generateContentStream`), cancellation, 30 s timeout, two retries with jittered backoff, friendly errors ("The assistant couldn't reply. Your message is saved.").
- Local limit counters in Firestore `usage/{uid_yyyyMMdd}` (increment-only rule) shown to the user; `PENDING BACKEND` for hard enforcement.
- Weekly AI report (Pro): generated on demand from the last 28 days of indices; stored in `users/{uid}/reports`.

E6. Recommendations (`recs/`)

Rule-based templates keyed by skin type/concern/season/Teen-mode, then optionally phrased by the coach. Data in `assets/recs.json`: ingredient class, purpose, how often, evidence tag, caution, "ask a clinician if...". No brand names. India-aware wording is not required; say "availability and prescription rules differ by country".

E7. Plan, habits, reminders

Habits (max 4 per day) from the plan; toggle persisted in Room; streak counts days with ≥ 1 habit done, **one rest day per week keeps the streak**; no XP, levels, ranks, or leaderboards. WorkManager schedules: weekly scan reminder, optional daily habit nudge, optional weekly report; honour quiet hours; reschedule on boot and on timezone change; cancel on sign-out/delete.

E8. Progress and history

Single source of truth = Room. Cloud sync (only when consent (c)/(sync) is on): upload scan *summaries* to Firestore and optionally the JPEG to Storage per retention policy; conflict rule: last-write-wins by `updatedAt`; tombstones for deletes; sync is idempotent and resumable (WorkManager, network-constrained).

E9. Photos and encryption

Photos are stored only per retention setting (default: not kept after analysis; results keep metrics only). When kept locally: JPEG saved to `filesDir/photos/` encrypted with AES-256-GCM using an Android Keystore key (write a small `PhotoVault` class using `Cipher` and `CipherOutputStream`; do not use deprecated crypto libraries). Re-encode every photo (apply EXIF rotation first) so no metadata survives. Gallery import uses `ActivityResultContracts.PickVisualMedia` (no storage permission). `FLAG_SECURE` on photo-bearing screens when the setting is on.

E10. Entitlements and billing

`BillingRepository` (Play Billing KTX) with product ids `lumora_pro_monthly`, `lumora_pro_yearly` (configurable), connection retry, `queryProductDetailsAsync`, `launchBillingFlow`, `PurchasesUpdatedListener`, acknowledge purchases, `queryPurchasesAsync` for restore. `EntitlementRepository` exposes `StateFlow<Entitlement(tier, source, expiresAt, verified)>`, cached in DataStore. `EntitlementGate.check(Feature)` is the only place limits are evaluated. Never hard-code prices. Real purchases require the app on a Play testing track; in debug builds provide a compile-time-gated "Simulate Pro" switch and a visible `UnverifiedBadge`. Document the future Cloud Function (`verifyPurchase` using the Google Play Developer API → writes `users/{uid}.entitlement`) as `PENDING BACKEND`; security rules already forbid client writes to `entitlement`.

F. KOTLIN / COMPOSE ARCHITECTURE

```
app/src/main/java/<pkg>/
  LumoraApp.kt (Application + AppContainer/Hilt)   MainActivity.kt
  core/        AppConfig, AiConfig, CaptureConfig, EntitlementPolicy, Result/Outcome, Dispatchers, Clock, Logger
    (no PII)
  data/
    local/     LumoraDatabase, dao/*, entity/*, datastore/*, PhotoVault
    remote/    FirebaseModule, AuthDataSource, FirestoreDataSource, StorageDataSource, AiDataSource (Firebase AI
     Logic)
    billing/   BillingRepository
    repo/      AuthRepository, ProfileRepository, ConsentRepository, ScanRepository, ChatRepository, PlanReposit
```

```
    ory,
                EntitlementRepository, ExportRepository, AccountRepository, SyncRepository
  domain/       model/*, usecase/* (CaptureQuality, MeasureFace, ScoreSkin, NoiseFloor, BuildContext, DeleteAccou
    nt, ExportData)
  face/         engine/*, measure/*, quality/*
  skin/         SkinStats, SkinScoreEngine
  coach/        CoachPrompts, CoachSession
  notifications/ Channels, ReminderScheduler(WorkManager), workers/*
  ui/           theme/, components/, nav/(Route, AppNavGraph), screens/<feature>/(Screen, ViewModel, UiState, Eve
    nts)
  util/         ImageUtils, ExifUtils, Permissions, Haptics
```

Patterns: each screen has `XViewModel : ViewModel` exposing `val uiState: StateFlow<XUiState>` and `onEvent(XEvent)`; one-off effects via `Channel`/`SharedFlow`; repositories return `Flow` or `suspend` with a `Outcome<T>` type (no thrown exceptions across layers); `viewModelScope` only; `collectAsStateWithLifecycle`; `Dispatchers` injected for tests.

G. DATA MODELS AND SCHEMA

Room: `ScanEntity(id PK, createdAt, kind, engineVersion, qualityScore, qualityFlagsJson, metricsJson, indicesJson, skinJson, photoFileId?, note, syncedAt?, updatedAt, deleted)`; `MetricNoiseEntity(metricId PK, sd, n)`; `ChatThreadEntity(id, title, updatedAt)`; `ChatMessageEntity(id, threadId FK, role, text, createdAt, reported)`; `HabitEntity(id, label, pillar, scheduleJson, active, orderIdx)`; `HabitLogEntity(date, habitId, done, PK(date, habitId))`; `ReminderPrefEntity(channel PK, enabled, timeMinutes)`; `ScanDraftEntity(id, stateJson)` for process death. Exported schema, migrations, `fallbackToDestructiveMigration` **not** allowed.

DataStore: theme, mirror-preview, haptics, hide-numbers, block-screenshots, app-lock, onboarding flags, consent cache, entitlement cache, last-seen legal versions.

Firestore:

```
users/{uid}                    {profile{displayName, birthYear?, skinType, concerns[], heightCm?, referenceRange
    Pref},
                                consent{version, at, onDevice, cloudAI, photoSync, analytics},
                                retention{mode, days}, entitlement{tier, source, expiresAt, verified},
                                deletionRequested?, createdAt, updatedAt}
users/{uid}/scans/{scanId}     {createdAt, updatedAt, kind, engineVersion, qualityScore, metrics{id:{v,sd,pos}},
     indices{}, skin{}, photoPath?}
users/{uid}/plan/current       {focusAreas[], actions[], updatedAt}
users/{uid}/habitLogs/{yyyyMMdd} {doneHabitIds[]}
users/{uid}/reports/{id}       {createdAt, windowDays, markdown}
users/{uid}/replyReports/{id}  {reason, note, replyText, createdAt}
usage/{uid_yyyyMMdd}           {aiSkin:int, coachMsgs:int}
```

Firebase Storage: `users/{uid}/scans/{scanId}.jpg` only.

H. AUTHENTICATION

Firebase Auth email/password (verification + reset) and Google via Credential Manager. Observe `AuthStateListener` as a `Flow<SessionState>`. Reauthenticate before delete/export/email change. Map every `FirebaseAuthException` to friendly text; avoid account enumeration on reset. Sign-out clears per section R. Never log tokens or emails.

I. PHOTO CAPTURE AND IMAGE SELECTION

CameraX lifecycle-bound use cases; front camera only for scan; handle rotation, mirroring, camera unavailable, permission denial (including "don't ask again"), lifecycle pause/resume, and Doze. Gallery via Photo Picker. All images decoded with `inSampleSize` to ≤ 1280 px long edge for analysis; bitmaps recycled; no full-resolution images held in ViewModels.

J. PRIVACY AND SECURITY REQUIREMENTS

Explicit, granular, revocable consent before any photo leaves the device; plain-language disclosure screen (N09/N33) listing exactly what is sent, to whom (Google Gemini via Firebase AI Logic), and why; user-controlled retention (default: do not keep photos); secure auth; App Check on all Firebase calls (Play Integrity provider for release, Debug provider for development); no secrets in code/assets; `android:allowBackup="false"` and `dataExtractionRules`/`fullBackupContent` excluding DB and photos; network security config forbidding cleartext; deny-by-default Firestore/Storage rules (section W); no unnecessary personal data (birth *year* only, optional); no face recognition; logs contain no PII; account and

data deletion; export; optional biometric app lock; no ads, no third-party trackers.

K. DEPENDENCY CHECKS AND FALLBACKS

Verify each compiles before building features on it: CameraX; ML Kit face-detection; ML Kit face-mesh-detection (beta) → fallback to face-detection contours; Firebase BoM with `firebase-auth`, `firebase-firestore`, `firebase-storage`, `firebase-ai`, `firebase-appcheck-playintegrity` + `-debug`; `androidx.credentials` + `googleid`; Play Billing KTX; Room (KSP); WorkManager; DataStore; Navigation Compose; `compose.ui.text.google.fonts`; biometric. If `firebase-ai` is unavailable under this environment, use the current Firebase AI Logic artifact that resolves and tell me the name. If Room's KSP fails, report and use a verified alternative. Do not add libraries that are not required.

L. LOADING / SUCCESS / FAILURE / EMPTY STATES

Every async screen has skeleton loading, success, empty, and error variants; every failure message states what happened, what is safe, and one action. Specific copy to implement: "We couldn't find a face. Choose a clear photo looking at the camera." · "More than one face is in this photo." · "This photo is too dark/blurry." · "Part of the face is cut off." · "The assistant couldn't reply. Your message is saved." · "You're offline. Scans and history still work; AI features resume when you're back online." Success toasts are brief and never block.

M. ERROR HANDLING MATRIX

No face / multi-face / low light / blur / head turned / eyes closed / expression / cropped; camera unavailable or in use; permission denied/permanently denied; AI timeout, safety block, malformed JSON (validate schema, fall back to on-device); network loss mid-call; Firestore permission denied (sign-out + message); App Check failure (explain "device could not be verified"); Billing unavailable/pending/cancelled/already owned; out of storage; keystore failure (offer to continue without keeping photos); process death mid-scan (resume from draft).

N. INTERNAL MODULES THAT MUST EXIST AS TESTABLE UNITS

`CaptureQualityEvaluator`, `OneEuroFilter`, `LandmarkAggregator` (median burst), `MeasurementCalculator`, `RangePositionMapper`, `NoiseFloorEvaluator`, `SkinStatsExtractor`, `SkinScoreEngine`, `ConsentPolicy`, `EntitlementGate`, `ContextBuilder`, `RetentionPolicyApplier`, `AccountDeleter`, `ExportBuilder`.

O. TESTING REQUIREMENTS (write and run; report only what actually ran)

Unit (JVM): measurement calculators with synthetic symmetric/asymmetric/tilted fixtures; range mapping; noise floor; quality score; One-Euro filter; median aggregation; skin score mapping and bounded adjustment; entitlement gate limits; consent policy; context builder excludes images/landmarks; retention applier; ViewModel state machines with fakes and `runTest`. **Instrumented / Compose UI:** Room migrations and DAOs; DataStore; navigation graph (every route reachable, Back behaviour); onboarding validation; consent switches; results list rendering; accessibility checks (touch target, content descriptions). **Firestore/Storage rules:** provide `rules-tests/` with emulator test cases (owner can read/write own docs; cannot read others; cannot write `entitlement`; oversize/non-JPEG upload denied; usage counters increment-only). **Manual test plan (print as `TESTING.md`):** first install, first launch, sign-up, sign-in, sign-out, Google sign-in, profile completion, camera permission (grant, deny, deny-forever), gallery permission-less picker, front scan, side scan, no-face, poor lighting, wrong head position, blurry image, AI timeout, airplane-mode mid-call, empty results, history, progress, coach, message failure, subscription state, restore, logout cleanup, account deletion, data deletion, app restart, process recreation (Developer options → Don't keep activities), small screen (360×640), large/tablet, foldable, font scale 200 %, TalkBack, dark mode, Back navigation from every screen. **Requires a real device** (the cloud emulator cannot provide them): real camera quality/auto-focus/exposure behaviour, live face-gate tuning, front-camera mirroring on several OEMs, Google sign-in with real accounts and SHA-1, Play Billing purchases/restore/pending, App Check Play Integrity, biometric prompt, notification delivery under Doze/battery optimisation, thermal/performance of live analysis, low-light accuracy. Do **not** mark any test as passed without executing it; mark the rest `NOT RUN (needs device)`.

P. ACCEPTANCE CRITERIA PER STAGE

Each stage compiles with zero errors, all stage unit tests pass, no TODO placeholders in shipped paths, every visible control does something real or is explicitly disabled with an explanation.

Q. PERFORMANCE

Live analysis ≤ 100 ms/frame on mid-range devices (downscale, reuse buffers, run off main thread), 10 fps analysis cap; cold start < 2 s on mid-range; LazyColumn keys; avoid recomposition storms (`remember`, stable state); baseline profile optional.

R. LOGOUT, DELETION AND CLEANUP

Sign out: cancel WorkManager jobs and notifications; clear Room tables for chats, scans, drafts (after confirming local-only data will be lost if not synced — offer "Export first"); delete the Keystore key and local photos; clear DataStore session keys; sign out of Firebase and Credential Manager (`clearCredentialState`). **Delete photos:** removes local photos and Storage objects, keeps metrics, updates `photoPath`. **Delete account:** reauth → mark `deletionRequested` → delete Storage objects → delete Firestore subcollections in paged batches → delete `usage` docs → delete local data and Keystore key → delete Firebase Auth user → sign out. Idempotent and resumable.

S. ACCESSIBILITY AND RESPONSIVE QUALITY CHECKLIST

TalkBack traversal logical; gate changes announced; no information by colour alone; text scales; touch targets ≥ 48 dp; dark theme; landscape; tablets (rail + two-pane); RTL-safe (`start/end`); `strings.xml` for all copy.

T. LEGAL AND DISCLOSURE CONTENT TO GENERATE (bundle as Markdown assets, versioned)

Privacy Policy, Terms, Methodology (auto-generated from the metric catalogue), Medical disclaimer ("Lumora is a cosmetic photo tool, not a medical device; it does not diagnose or screen for any condition"). They must match actual behaviour: what is on-device, what is sent to Google Gemini via Firebase AI Logic only with consent, retention options, deletion, export, no analytics by default, ages 13+ with Teen mode, no face recognition. Include a placeholder `[Operator name and contact — fill in before release]` instead of inventing a company.

U. QUALITY SELF-CHECK BEFORE YOU FINISH

Re-read this prompt and confirm: every screen N01–N41 exists and is reachable; no dead buttons; no features only simulated (list any `PENDING BACKEND`); privacy rules honoured (no uploads without consent; no secrets in the APK); no tier/attractiveness/dimorphism content; every error path handled; tests honestly reported. List deviations and why.

V. IMPLEMENTATION STAGES (compile and test at the end of each)

Stage 1 — Foundations. Project audit, version catalogue, packages, theme/typography/components, `AppContainer`, Room/DataStore, navigation skeleton with all routes as real (even if minimal) screens, string resources, error/loading components, unit test setup. **Stage 2 — Account and onboarding.** Firebase Auth (email + Google), N01–N10, profile persistence (Firestore + local), consent model, session routing, rules file, notification rationale. **Stage 3 — Capture and face engine.** CameraX, permissions, gates, burst aggregation, engines (+ fallback), `PhotoVault`, gallery import, review, processing; unit tests for quality/aggregation. **Stage 4 — Measurements, results, history, compare.** Measurement engine + catalogue + reference bands JSON, results/detail/overlay, Room timeline, Progress, Compare with noise floor, Today screen basics, Scan details, delete scan; Methodology generator. **Stage 5 — Skin and side profile.** On-device skin stats, Firebase AI Logic structured call, `SkinScoreEngine`, skin results, side-profile (beta, unscored). **Stage 6 — Coach, plan, routines, reminders.** Chat (streaming), history, report-reply, limits, plan/habits/routine builder, weekly check-in, WorkManager reminders, recommendations data. **Stage 7 — Entitlements and billing.** `EntitlementPolicy`, gates everywhere, paywall, restore, subscription screen, `UnverifiedBadge`, debug-only simulate-Pro. **Stage 8 — Privacy centre and data rights.** Retention policies, delete photos, export, delete account, app lock, screenshot block, sign-out cleanup, cloud sync (opt-in), legal docs. **Stage 9 — Polish and verification.** Accessibility pass, tablet/foldable layouts, animations, performance, rules tests, instrumented tests, `TESTING.md`, final self-check, release-build check

(R8/ProGuard keep rules for Firebase, ML Kit, Billing), `README.md` with setup.

W. FIREBASE SETUP REQUIREMENTS

Typically automatic in AI Studio (confirm on first run and tell me what you could not confirm): Firebase project link, Auth enablement, Firestore database creation, Firebase AI Logic enablement. **Manual Firebase Console steps — print this checklist at the end of Stage 2 and again at the end:**

1. Authentication → enable **Email/Password** and **Google**; add this app's **SHA-1 and SHA-256** (debug and release); download updated `google-services.json` if prompted.
2. Firestore → deploy the rules below (production mode). Storage → deploy the Storage rules below.
3. **App Check** → register the Android app with **Play Integrity** (release) and add a **debug token** for development; turn on enforcement for Firestore, Storage and Firebase AI Logic after verifying traffic.
4. **Firebase AI Logic** → Gemini Developer API enabled; set a per-user rate limit; add budget alerts in Google Cloud.
5. **Play Console** → create subscriptions `lumora_pro_monthly`, `lumora_pro_yearly`; upload to an internal testing track; add licence testers.
6. Fill the operator name/contact placeholder in legal documents.

Firestore rules (write to `firestore.rules` and print):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() { return request.auth != null; }
    function isOwner(uid) { return signedIn() && request.auth.uid == uid; }
    function noEntitlementWrite() {
      return !request.resource.data.diff(resource.data).affectedKeys().hasAny(['entitlement']);
    }

    match /users/{uid} {
      allow read: if isOwner(uid);
      allow create: if isOwner(uid) && !request.resource.data.keys().hasAny(['entitlement']);
      allow update: if isOwner(uid) && noEntitlementWrite();
      allow delete: if isOwner(uid);

      match /scans/{scanId}        { allow read, delete: if isOwner(uid);
                                     allow create, update: if isOwner(uid)
                                       && request.resource.data.keys().hasOnly(['createdAt','updatedAt','kind','
    engineVersion','qualityScore','metrics','indices','skin','photoPath'])
                                       && request.resource.data.kind in ['front','side']; }
      match /plan/{doc}            { allow read, write: if isOwner(uid); }
      match /habitLogs/{day}       { allow read, write: if isOwner(uid); }
      match /reports/{id}          { allow read, write: if isOwner(uid); }
      match /replyReports/{id}     { allow create: if isOwner(uid) && request.resource.data.keys().hasOnly(['rea
    son','note','replyText','createdAt']);
                                     allow read, delete: if isOwner(uid); }
    }

    match /usage/{docId} {
      allow read: if signedIn() && docId.matches(request.auth.uid + '_.*');
      allow create: if signedIn() && docId.matches(request.auth.uid + '_.*') && request.resource.data.keys().has
    Only(['aiSkin','coachMsgs']);
      allow update: if signedIn() && docId.matches(request.auth.uid + '_.*')
        && request.resource.data.aiSkin >= resource.data.aiSkin && request.resource.data.aiSkin <= resource.data
    .aiSkin + 1
        && request.resource.data.coachMsgs >= resource.data.coachMsgs && request.resource.data.coachMsgs <= reso
    urce.data.coachMsgs + 1;
      allow delete: if signedIn() && docId.matches(request.auth.uid + '_.*');
    }
    match /{document=**} { allow read, write: if false; }
  }
}
```

(Review these rules, fix any syntax or logic problems against the current rules language, add field-type validation for `users/{uid}` and `scans`, and write emulator tests. These rules give *soft* quota control only; hard enforcement is `PENDING BACKEND`.)

Storage rules (write to `storage.rules` and print):

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /users/{uid}/scans/{file} {
      allow read, delete: if request.auth != null && request.auth.uid == uid;
```

```
      allow write: if request.auth != null && request.auth.uid == uid
        && request.resource.size < 2 * 1024 * 1024
        && request.resource.contentType == 'image/jpeg';
    }
    match /{allPaths=**} { allow read, write: if false; }
  }
}
```

X. EXTERNAL BACKEND (only if essential, and not generated here)

Do **not** create a server project. List, as documentation in `docs/BACKEND_TODO.md`: (1) callable Cloud Function `verifyPurchase` (Google Play Developer API) writing `users/{uid}.entitlement`; (2) scheduled/Callable function enforcing daily AI quotas; (3) optional Firebase "Delete User Data" extension as a deletion safety net. State clearly that until they exist the app shows `UnverifiedBadge` and soft quotas.

Y. BUILD, DEPLOYMENT AND APK TESTING

Ensure `./gradlew assembleDebug` and `./gradlew testDebugUnitTest` pass; set `minSdk 26`, `targetSdk` and `compileSdk` to the latest stable your environment supports; enable R8 for release with correct keep rules; produce a debug APK for sideloading; tell me exactly: how to install the APK on a real device, how to add the debug App Check token, how to test Google sign-in (SHA-1), how to test billing via an internal test track, and what could not be tested in the cloud emulator. End with a final report: stages completed, tests run (with real results), `PENDING BACKEND` items, known limitations, and the manual steps still required from me.